

スキルシート

- このスキルシートのリンク - <https://github.com/sugurutakahashi-1234/skills-sheet>

基本情報

- 所在: 東京
- 現在のポジション: AI コンサルタント / FDE (Forward Deployed Engineer) / CTO
- エンジニア歴: 10年目 (2017年～)
- 外部リンク:
 - GitHub - <https://github.com/sugurutakahashi-1234>
 - X - https://x.com/suguru_takaha4
 - Qiita - <https://qiita.com/sugurutakahashi12345>
 - Zenn - <https://zenn.dev/ikuraikura>

サマリー

エンジニア歴 10年目。Web・モバイルのフロントエンドからサーバーサイド、マルチクラウド・IaC までの実装経験あり。現在は創業フェーズの [株式会社ZENSHIN](#) の CTO を務めながら、AI コンサルタント / FDE として AI 活用の相談・提案（プリセールス）から、RAG・MCP・AI エージェント基盤の設計・実装・導入、プロダクト開発の推進までを一気通貫で担当。

強み

- AI コンサルティング～FDE**
 - 経営層から実装担当者まで、相手に合わせた AI 活用の相談・提案（プリセールス）を担当。相談・提案から実装・導入までを一気通貫で支援した経験
- AI システム開発（RAG / MCP / AI エージェント）**
 - 業務プロセスに AI を組み込んだシステムを、設計から実装・実運用まで単独でフルスタック開発できる。実際に人材マッチングや開発案件の自動評価のシステムを Cloudflare 上に開発し、非エンジニアの営業が Claude Code / Codex で実業務を回せる状態まで導入した経験
- CTO 経験**
 - 創業フェーズの AI コンサルティング会社で提案活動・AI システム開発・コーポレートサイト・社内インフラを単独で推進
- 0 → 1 リードエンジニア経験（5件）**
 - アーキテクチャ選定・CI/CD 構築・チーム運用の設計など、プロジェクト立ち上げの技術リードを担当（現職 CTO でのプロダクト立ち上げを含む）
- KPI 起点のグロース施策**
 - グロースフェーズのアプリ開発で、ビジネス KPI（ARPU・LTV・CTR）を基点とした施策の企画・A/B テスト検証・実装までを一気通貫で推進
- フルスタック実装力**
 - iOS・Android・Web のフロントエンド（Swift / React Native / Flutter / Next.js / Astro）からサーバーサイド、マルチクラウド・IaC（Terraform × GCP / Cloudflare / AWS）まで単独で実装可能
- OSS 開発・技術発信**
 - 自作 OSS（TypeScript 製 CLI ツール）を npm / Homebrew / GitHub Actions Marketplace で配布し、Zenn で技術記事を継続発信

スキルセット

AI コンサルティング / AI システム開発

- **AI コンサルティング～FDE**
 - 月額顧問型「AI 相談サービス」による顧客企業の AI 活用支援
 - LLM ツール（ChatGPT / Claude / Gemini）の使い分け指導、プロンプト・ツール設定の改善
 - MCP 化・Skills 設計の方針策定、システム化の進め方の伴走支援
 - コンサルティングにとどまらず、FDE（Forward Deployed Engineer）として実装・導入までを一気通貫で支援
- **AI システム開発（RAG / MCP / エージェント基盤）**
 - Cloudflare Workers + Vectorize + Workers AI による RAG・ベクトル検索システムの構築
 - Better Auth（OAuth）認証付き MCP サーバー（40+ ツール）の設計・実装と、非エンジニア（営業）が Claude Code / Codex から実業務を行える仕組みの実運用
 - Cloudflare Workflows / Queues による LLM 多段パイプラインの構築、Claude Code Skills + 定期ルーチンによる業務自動化
 - ユースケース別の LLM モデル選定（Kimi K2 / gpt-oss / GLM / Llama、精度・レイテンシ・コストのバランスで使い分け）
 - プロンプトのバージョン管理と、本番データでの「AI 判定 vs 人間判断」の乖離分析によるデータ駆動のプロンプト改善運用
- **AI エージェント / コーディング支援ツールの活用**
 - Claude Code / Codex を中心に、Skills・CLAUDE.md / AGENTS.md・MCP・定期ルーチンを整備し、コーディング支援にとどまらず業務自動化までエージェントを活用
 - GitHub Actions による Claude Code 自動レビュー、Claude Code Managed Agents（Web 版）の運用
 - Cloudflare MCP（本番 Worker のログ調査・D1 / KV の参照・デプロイ調査・公式ドキュメント検索）、gcloud MCP（GCP 運用）、Slack MCP（運用調査）を日常の開発・運用に組み込み
 - Figma MCP（デザインからの UI 実装）、Chrome DevTools MCP / Playwright MCP（UI の自動検証・本番の認証越し動作確認）、AWS MCP Servers（予算見積もり・構成図・ベストプラクティス収集）の活用

プロジェクト推進 / チーム開発

- **PdM / ビジネスサイド**
 - 顧客ヒアリングによる課題定義、提案書・見積書の作成、投資対効果まで含めた意思決定支援
 - ビジネス KPI（ARPU・LTV・CTR）を基点としたグロース施策の企画・A/B テスト検証・実装までの一気通貫の推進
- **リードエンジニア**
 - 0 → 1 フェーズでのプロジェクト立ち上げを担当（5件）
 - アーキテクチャやライブラリの選定を含むプロジェクト立ち上げにおける技術選定の経験
 - プロジェクトのランチ戦略、CI/CD 環境の設計・構築
 - スクラムボードの設計・運用の経験
 - PdM、デザイナー、サーバーサイドエンジニアとの仕様調整の経験
- **アジャイル開発**
 - スクラム、カンバンなどの手法を用いたアジャイル開発の経験
 - Jira, Confluence, GitHub Projects, Trello, Zenhub, Notion, Backlog, Linear などのプロジェクト管理ツールの経験
- **Scrum Master**
 - Scrum Master としてスプリントのプランニング、リファインメント、レトロスペクティブの実施やスクラムボードの設計の経験

開発

- **インフラ / IaC**
 - Terraform によるマルチクラウド（GCP / Cloudflare / AWS）の IaC 構築・運用
 - GCS Remote Backend による Terraform state の一元管理
 - Workload Identity Federation (OIDC) による GitHub Actions → GCP の鍵レス認証（Service Account Key の発行不要）
 - Cloudflare Access（Zero Trust）+ Google OAuth による環境へのドメイン認証アクセス制限
 - AWS Organizations によるマルチアカウント環境（prod / stg / dev）の構築
- **クラウド連携**
 - **Firebase**
 - Authentication, Firestore, Storage, Analytics, Crashlytics, App Distribution, Remote Config などを利用した実装
 - **AWS**
 - AppSync, Cognito, S3, Pinpoint などの AWS サービスを Amplify SDK 経由での活用
 - **Google Cloud**
 - Cloud Storage, Cloud Run, Google Maps API, Google Cloud API Key の活用
- **サーバーサイド**
 - OpenAPI (Swagger) でのインターフェース設計
 - REST API / RPC の設計・開発（TypeScript: Hono / oRPC, PHP）
 - RDB のテーブル設計
- **React Native (Expo) / Next.js**
 - Expo による iOS / Android / Web のクロスプラットフォーム開発
 - Monorepo 構成によるモバイル・Web 間のコード共有
 - TanStack Query によるデータフェッチング・キャッシング
- **Swift**
 - SwiftUI, Combine, Swift Concurrency を活用した宣言的 UI・非同期処理の実装
 - Swift Package Manager を使用したマルチモジュール構成の設計・実装
- **Flutter**
 - Riverpod + Hooks を用いた状態管理
 - go_router によるルーティング、OpenAPI Generator による API クライアント自動生成
- **アーキテクチャ**
 - Clean Architecture, VIPER, MVVM, Redux などプロジェクト特性に応じた設計選定
 - レガシーコードのアーキテクチャ刷新・段階的リファクタリング
- **CI/CD**
 - Xcode Cloud, Bitrise, GitHub Actions による TestFlight / Firebase App Distribution への自動配信
 - Renovate による依存関係の自動更新
 - release-please によるリリース自動化
- **ハードウェア連携**
 - ドローン SDK によるリアルタイム飛行制御
 - インカメラ連携による視線分析・感情分析の実装
- **OSS 開発**
 - [ai-chat-md-export](#):
 - ChatGPT や Claude のチャット履歴を Markdown 変換するツール（npm, Homebrew）
 - [mermaid-markdown-wrap](#):
 - Mermaid ファイルを Markdown コードブロックでラップするツール（npm, GitHub Actions Marketplace）

- [issue-linker](#):
 - コミットメッセージ、ブランチ名から GitHub Issue への参照を検証するツール (npm, GitHub Actions Marketplace)
- [readme-i18n-sentinel](#):
 - README の翻訳版と原文の構造的ー貫性を検証するツール (npm)

技術スタック

- **AI コンサルティング**
 - 月額顧問型 AI 相談サービス (AI 活用提案, LLM ツール使い分け, プロンプト改善, MCP 化 / Skills 設計方針), FDE としての実装・導入支援
- **AI システム開発**
 - RAG / Embedding / ベクトル検索 (Cloudflare Vectorize, Workers AI), MCP サーバー開発 (@modelcontextprotocol/sdk, Better Auth OAuth), Vision LLM, Cloudflare Workflows / Queues, ユースケース別 LLM モデル選定 (Kimi K2 / gpt-oss / GLM / Llama), プロンプト運用 (バージョン管理・本番データでの継続改善)
- **生成 AI 活用**
 - Claude Code (Skills / MCP / エージェント運用 / GitHub Actions 自動レビュー), Codex CLI, Claude Code Managed Agents
- **言語**
 - TypeScript, Swift, Dart, PHP, Java
- **クラウド**
 - AWS (Amplify, AppSync, Cognito, S3, Route 53), Google Cloud (Cloud Run, GCS, IAM), Cloudflare (Workers, Pages, D1, Vectorize, Workers AI, AI Gateway, Workflows, Queues, R2, KV, Browser Rendering, Email Routing, DNS, Turnstile, Access), Firebase
- **インフラ / IaC**
 - Terraform, tfint, dotenvx, OIDC
- **サーバーサイド**
 - REST API 設計, GraphQL 設計, Node.js, Hono, oRPC, PHP, Java
- **データベース**
 - RDB のテーブル設計, Prisma ORM, Drizzle ORM, PostgreSQL, MySQL, SQLite (Cloudflare D1), Firestore, ベクトル DB (Cloudflare Vectorize)
- **認証 / 認可**
 - OAuth 2.0, JWT, Better Auth, Cognito, Firebase Authentication, Cloudflare Access, Workload Identity Federation (OIDC)
- **Web アプリ開発**
 - React, Next.js, Astro, TanStack Query, Tailwind CSS
- **モバイルアプリ開発**
 - React Native (Expo), Swift (SwiftUI, UIKit), Flutter
- **アーキテクチャ**
 - モノレポ設計 (Bun workspaces), Clean Architecture, レイヤーDアーキテクチャ, MVVM, Redux, Riverpod
- **テスト**
 - Vitest, Playwright, Swift Testing, XCTest, Quick/Nimble, Maestro, Storybook, @axe-core/playwright
- **コード品質**

- Biome, oxlint, oxfmt, ESLint, Prettier, knip, lefthook, husky, Renovate
- **CI/CD**
 - Xcode Cloud, GitHub Actions, Bitrise, release-please, Wrangler
- **SEO / パフォーマンス**
 - Core Web Vitals, Lighthouse, JSON-LD, OGP, satori, Google Search Console
- **プロジェクト管理**
 - Scrum Master 経験, アジャイル開発 (Jira, GitHub Projects, Zenhub, Linear)

職務経歴

株式会社ZENSHIN (2026年04月 - 現在)

- 2026年
 - [No.13] [株式会社ZENSHIN](#) - CTO / AI コンサルタント / FDE (AI コンサルティング会社)
 - AI コンサルティング / 人材マッチング・案件自動評価などの社内 AI システム開発 (RAG・MCP・AI エージェント) / コーポレートサイト・資料共有基盤の開発 / 社内インフラ管理

フリーランス (2021年07月 - 2026年03月)

- 2025年
 - [No.12] ショートドラマアプリ開発 - フロントエンドエンジニア (React Native / Next.js) (エンタメ業界 G社)
 - [No.11] OSS 開発 - 個人開発 (TypeScript)
 - [No.10] NFT ゲームアプリ開発 - Flutter リードエンジニア (WEB3 特化 Sler F社)
- 2024年
 - [No.9] SNS アプリ開発 - iOS リードエンジニア (Sler E社)
 - [No.8] Clean Architecture ベースの新アーキテクチャの考案 - 個人開発 (Swift)
- 2023年
 - [No.7] マーケティングリサーチアプリ開発 - iOS リードエンジニア (スタートアップ D社)
- 2022年
 - [No.6] ファンクラブアプリ開発 - iOS エンジニア (メガベンチャー C社)
- 2021年
 - [No.5] ドローン制御アプリ開発 - iOS エンジニア (ドローンベンチャー B社)

外資系ITコンサル A社 (2017年04月 - 2021年06月)

- 2021年
 - [No.4] ショッピングアプリ開発 - Flutter エンジニア
 - [No.3] 飲食店管理アプリ開発 - iOS リードエンジニア
- 2018年 - 2020年
 - [No.2] クレジットカードアプリ iOS アプリ開発 - iOS エンジニア
 - [No.1] クレジットカードアプリ API 開発 - サーバーサイドエンジニア

※ 各案件の詳細は以下のプルダウンから確認可能。

[No.13] 株式会社ZENSHIN - CTO / AI コンサルタント / FDE (AI コンサルティング会社)

チーム体制

- CTO として、AI コンサルティング / 提案活動（FDE）から社内 AI プロダクト・コーポレートサイト・社内インフラの開発・運用までを一人で担当

案件概要・担当業務

- 創業フェーズの株式会社ZENSHIN の CTO として、以下 6 つの業務を並行して推進
 - 業務 1: AI コンサルティング / 提案活動 — AI 活用相談・システム構築提案を推進し、FDE として実装・導入まで自ら支援
 - 業務 2: AI マッチングシステム開発 — Cloudflare フルスタック構成での RAG / MCP / AI エージェント基盤の開発・運用
 - 業務 3: コーポレートサイト開発 — <https://www.zenshin-inc.co.jp/> の設計・構築・運用
 - 業務 4: 社内インフラ管理 — Terraform によるマルチクラウド IaC
 - 業務 5: AI 案件選別システム開発 — 外部案件を LLM で自動評価し、応募判断を支援する社内 AI システムの開発・運用
 - 業務 6: 社外向け資料共有基盤の開発 — Cloudflare Access 認証付きの顧客向け資料配信基盤の開発・運用

業務 1: AI コンサルティング / 提案活動（FDE）

概要

- システム構築案件の提案活動と、月額顧問型の「[AI 相談サービス](#)」による AI コンサルティングを一体で推進
- システム構築案件では、AI ツールを活用したデモ・プロトタイプ作成と提案書の作成を主導し、CEO や営業と同居してお客様との要件ヒアリングから提案までを技術面でリード
- AI 相談サービスでは、経営者・DX 推進担当者・社内システム担当者を対象に、月 2～4 回のオンライン相談形式で伴走
- 相談・提案で終わらせず、FDE（Forward Deployed Engineer）として実装・導入まで自ら支援

経験した技術

- AI ツールを活用したデモ / プロトタイプ / 提案書・見積書の作成
 - v0 (Vercel) / Replit / Lovable / Google AI Studio による UI・アプリの高速プロトタイピング
 - Claude Code / Codex による実装レベルのデモ作成
 - Google Stitch / Figma によるデザインモックアップ作成
 - プロトタイプを組み込んだ提案書・見積書の作成
- AI コンサルティング（AI 相談サービス）
 - 現状ヒアリングと課題整理、業務に適した AI 活用方法の提案
 - ChatGPT / Claude / Gemini などの LLM ツールの使い分け指導、プロンプト・ツール設定の改善支援
 - MCP 化や Claude Code Skills 設計の方針策定支援

取り組み・貢献

- 課題起点のヒアリング重視の提案
 - お客様の提示するシステム要件をそのまま受けず、「何に困っているか」「理想の姿」を深掘りするヒアリングを実施
 - お客様の当初の構成案が最適でないケースも多いため、課題定義から提案し直すことを心がけた
- 提案前の技術検証と実装先行
 - AI 開発ツールを活用し、商談前の段階で 1～2 日でプロトタイプを作成して実現可能性を事前報告
 - 実現可能性・期間・想定工数の見積もりも事前に詰めておくことで、精度・具体性の高い提案を実現
 - 商談中に不明点があればその場で AI を使って調査し、回答まで完結させた
- CEO・営業との提案方針のすり合わせ
 - 提案内容・方針・見積もりを事前に CEO・営業と合意してから提案に臨む

- 設計まで完了させた上で A / B / C の複数案を準備し、お客様の反応に応じて即座に出し分け
- **提案書・見積書の作成方針**
 - お客様の課題がどう解決されるかをシンプルに伝えることを重視
 - 自社予算での費用対効果（投資回収の見込み）まで含めて、意思決定材料となる提案書・見積書を作成
- **実践知に基づくコンサルティング**
 - 自社の AI マッチングシステム開発・AI エージェント運用で日々実践した知見を、そのままコンサルティングに還元
 - 経営層には投資対効果・方針レベル、実装担当者にはプロンプト・ツール設定レベルと、相談者のレイヤーに応じて支援内容を出し分け
- **コンサルから実装まで一気通貫の FDE スタイル**
 - 助言・提案で終わらず、MCP サーバー構築・Claude Code Skills 整備・AI ワークフロー構築まで自ら実装して顧客に届けるスタイルで支援を推進
 - 自社の AI マッチングシステムを単独でフルスタック開発・運用しており、提案の実現可能性と導入スピードを担保

業務 2: AI マッチングシステム開発（Cloudflare フルスタック AI 基盤）

概要

- Excel の手作業・キーワード検索に依存していた SES 事業のマッチング業務（要員 × 案件）を AI システム化し、Cloudflare Workers 上にフルスタック構成で単独開発・運用
- LINE で届く非構造的要員・案件情報を LLM で構造化し、ベクトル検索（RAG）でマッチング候補を抽出、AI エージェントが採点する多層パイプラインを構築
- IT 知識のない営業メンバーが、OAuth 認証付きの MCP 接続を通じて Claude Code / Codex からシステムを直接操作し、マッチングした人材の紹介までを実業務として運用
- 「AI にチャットで聞く」で終わらず、業務プロセスそのものに AI を組み込んだ業務システムとして設計

経験した技術

- **RAG / Embedding / ベクトル検索**
 - Workers AI (bge-m3, 1024 次元・多言語対応) による Embedding 生成と Cloudflare Vectorize によるコサイン類似度検索
 - メタデータフィルタによる検索母集団の絞り込みと、検索結果の D1 保存によるマッチング候補管理
 - ベクトル検索（一次絞り込み）→ AI エージェント採点（二次精査）の多層マッチングパイプライン設計
 - Embedding 粒度（entity 単位 vs project 単位）のコスト・精度トレードオフ検証と再設計
- **MCP サーバー開発**
 - @modelcontextprotocol/sdk による 40+ ツールを持つ MCP サーバーの実装（Cloudflare Workers 上）
 - Better Auth (Google OAuth) による MCP OAuth 認証・セッション管理で、業務データへのアクセスをユーザー単位でセキュアに制御
 - Claude Code / Codex の両 AI エージェントから同一の MCP サーバーを利用できるマルチエージェント対応（スキル定義も両エージェント向けに整備）
- **Cloudflare AI / サーバーレス基盤**
 - AI Gateway 経由の LLM 統一呼び出し（構造化抽出・Vision 解析・Embedding を同一バインディングで管理）
 - Cloudflare Workflows による durable execution (LLM 抽出 → 構造化 → Embedding → マッチング検索の step 単位リトライ)
 - Cloudflare Queues + DLQ による非同期マッチング検索、R2 によるベクトルデータ退避（Queue メッセージサイズ上限対策）
 - Browser Rendering + Puppeteer によるスキルシート PDF の画像化 → Vision LLM でのスキルシート解析

- D1 (SQLite) + Drizzle ORM による 100+ マイグレーション管理、KV によるアクセストークンキャッシュ
- Cron Triggers による定期処理（受信メッセージのグルーピング、日次レポート、監視 watchdog）
- ユースケース別の LLM モデル選定
 - Kimi K2 系 / gpt-oss 系 / GLM 系 / Llama 系など Workers AI 提供モデルを実タスクで比較検証
 - 「常に最高性能のモデルを使う」のではなく、構造化抽出・Vision 解析・Embedding などのシーンごとに精度・レイテンシ・コストのバランスでモデルを使い分け
 - モデル定数の一元管理により、用途別のモデル差し替えを容易にする設計
- AI エージェント運用基盤（Claude Code Skills）
 - 33 の Claude Code スキル + 定期ルーチンによる業務自動化（要員・案件の取り込み、マッチング採点、Slack への日次レポート配信）
 - CLAUDE.md / SKILL.md による判断ルール・業務フローの SSoT 化（Claude Code / Codex の両エージェント対応）
- フルスタック開発
 - Hono + oRPC による型安全な API、React 19 + TanStack Router / TanStack Query による SPA（Workers Static Assets で同一オリジン配信）
 - LINE Messaging API（Webhook / Push）、Slack API、Google Drive API との連携
 - Playwright E2E（隔離 D1 + 実 API モード）、Storybook + MSW、Bun workspace によるモノレポ管理

取り組み・貢献

- Excel 手作業のシステム化と「キーワード検索を超える」マッチングの実現
 - 従来は Excel 台帳の手作業と「Java」などのキーワード一致検索に依存していたマッチング業務を、構造化データとベクトル検索によるシステムへ置き換え
 - スキルシートの内容まで LLM で構造化し、要約文の Embedding（RAG）による意味ベースの類似検索を導入することで、キーワード一致ではヒットしない要員 × 案件のマッチングを発見できるように
- 非エンジニアの営業が AI エージェントで実業務を回す仕組みの実現
 - Better Auth の OAuth 認証によるセキュアな MCP 接続を整備し、IT 知識のない営業メンバーが Claude Code / Codex から自然言語で要員検索・マッチング確認・人材紹介までの実業務を完結できる体制を構築
 - MCP ツール設計・スキル整備・権限制御により、エンジニアを介さず営業自身が AI エージェントを日常業務で使いこなす状態を実現
- ユースケース別 LLM モデル選定によるコスト最適化
 - 複数系統のモデル（Kimi K2 / gpt-oss / GLM / Llama）を比較検証し、シーンごとにコストパフォーマンスの良いモデルを選定することで、精度と運用コストを両立
- 業務プロセス全体への AI 組み込み・自動化
 - LINE で届く営業メールの転送テキスト・添付スキルシートを、LLM 構造化 → ベクトル検索 → AI 採点まで自動処理する仕組みを構築
 - スキル + MCP + 定期ルーチンの組み合わせで、朝のマッチングレポート配信や要員の自動採点など、人手を介さない定常運用を実現
- 低コスト・高信頼な運用設計
 - インフラを Cloudflare のみで完結させ、小規模チームでも維持できる運用コストを実現
 - サイレント障害が起きていた非同期処理を Workflows の durable execution へ移行して根治するなど、信頼性を継続的に改善

業務 3: コーポレートサイト開発（Astro + Cloudflare Pages）

概要

- 株式会社ZENSHIN のコーポレートサイト (<https://www.zenshin-inc.co.jp/>) を Astro + Cloudflare Pages 構成で設計・構築・運用

経験した技術

- **Astro / Cloudflare Pages**
 - Astro 6 による SSG + Cloudflare Pages Functions による問い合わせ API の実装
 - Tailwind CSS v4 による UI 実装、ブランドカラーの設計
 - Astro Content Collections による blog / news / works / services のコンテンツ管理
 - Cloudflare Turnstile + Slack Bot Token による問い合わせフォームのスパム対策と通知連携
 - `astro:env` による下書き記事の環境別可視化
- **SEO / 構造化データ / Core Web Vitals**
 - Schema.org (Organization / ProfessionalService / BreadcrumbList) の JSON-LD 埋め込み
 - OGP / Twitter Card / canonical URL / `robots: noindex` のページ別制御
 - satori + sharp による OG 画像の動的生成 (記事ごとに 1200x630 PNG)
 - `@astrojs/sitemap` による sitemap.xml 自動生成
 - CSS インライン化と画像最適化による LCP 改善
 - Google Search Console / PageSpeed Insights による計測と継続的改善
- **CI/CD / テスト / 品質**
 - GitHub Actions による Preview / Staging / Snapshot / Production の多段デプロイパイプライン
 - `release-please` による Conventional Commits ベースの自動リリース
 - Vitest / Playwright / `@axe-core/playwright` による単体・E2E・アクセシビリティ検証
 - `oxlint` / `oxfmt` / `knip` / `lefthook` による静的解析・フォーマット自動化
- **生成 AI 活用**
 - Claude Code Managed Agents (Web 版 Claude Code) のセットアップ
 - Chrome DevTools MCP を活用したブラウザ動作検証
 - GitHub Actions による Claude Code 自動レビューの仕組み

取り組み・貢献

- **CEO・経営陣と連携したサイト構築**
 - 掲載内容・デザイン方針を CEO や経営陣と相談しながら決定
 - Staging 環境を整備し、段階的に経営陣の意見を取り込む CI/CD サイクルで開発を進めた
- **SEO / Core Web Vitals 改善**
 - CSS インライン化・画像最適化・構造化データ実装により Lighthouse 100/100 を達成・維持
 - Google Search Console と PageSpeed Insights をダッシュボードとして運用し、継続的に改善
- **技術ブログの立ち上げ・運営による採用・ブランディング貢献**
 - タグ / 月別アーカイブ、記事ごとの OGP 自動生成、執筆規約を整備
 - 技術選定・CMS 選定・Astro OGP 生成などの記事を自ら執筆し、採用・ブランディングに活用

業務 4: 社内インフラ管理 (Terraform マルチクラウド)

概要

- GCP / Cloudflare / AWS を横断管理する Terraform モノレポを構築・運用
- コーポレートサイトのホスティング基盤、DNS、Zero Trust アクセス制限、アカウント・通知設定などを IaC 化

経験した技術

- **Terraform / マルチクラウド**
 - GCP / Cloudflare / AWS を対象としたマルチモジュール構成の IaC 管理
 - GCS Remote Backend による Terraform state の一元管理
 - Workload Identity Federation (OIDC) による GitHub Actions → GCP の鍵レス認証

- Cloudflare Access (Zero Trust) + Google OAuth による Staging 環境へのアクセス制限
- dotenvx による各モジュールの `.env` 暗号化コミットと CI 復号
- **Cloudflare のセキュリティ / パフォーマンス機能**
 - Smart Tiered Cache / 0-RTT 接続再開 / Speed Brain / Page Shield の Terraform 管理化
 - apex→www 301 Redirect Rules による URL 正規化
 - WAF Custom Rule の導入検討と「現時点では設定しない」意思決定の文書化
- **CI/CD (Terraform) / ガバナンス**
 - GitHub Actions による PR 時の `terraform plan` 自動実行と `tfcmmt` による PR コメント
 - tfllint + lefthook による pre-commit / pre-push の自動検査
 - Renovate によるプロバイダ・依存の自動更新
 - Gmail フィルタ + Slack Email App による通知の Slack 集約
 - GitHub Organization のチーム構成・権限設計

取り組み・貢献

- **属人化の排除と継続運用の仕組み化**
 - 属人的になりがちな社内インフラ構成を徹底的に Terraform で IaC 化
 - Terraform で表現できない設定は README に明示し、CTO が不在でも運用が回る仕組みを整備
- **インフラ 100% IaC 化による安全なワークフロー**
 - コード変更 → PR → `terraform plan` CI → マージ → ローカル `apply` の一連のフローを確立
- **DNS 移行プロジェクトの遂行**
 - AWS Route 53 → Cloudflare DNS への段階的移行を Phase 管理で実施し、無停止で本番切替
- **技術的意思決定の文書化**
 - 採用しなかった選択肢 (WAF、Cache Rules など) とその理由もリポジトリに残す文化を形成

業務 5: AI 案件選別システム開発 (Cloudflare + LLM)

概要

- 外部の受発注マッチングプラットフォームから届く新着案件を LLM で自動評価し、応募可否の判断材料 (スコア・判定理由・応募文ドラフト) を Slack / Web に配信する社内システムを単独で設計・開発・運用
- 営業・経営層の「どの案件に応募すべきか」の判断を AI で支援し、最終判断のみ人間が行う設計

経験した技術

- **LLM パイプライン**
 - Slack Events → Cloudflare Queues (DLQ 付き) → Workflows (durable execution) の 3 段非同期パイプラインによる案件処理
 - 案件ページのスクレイピングと決定論的な DOM パース (構造化に AI を使わずコスト・不確実性を排除)、セッション Cookie の KV キャッシュ
 - Workers AI (Kimi K2.6) + AI Gateway による 1 コールでのスコアリング・応募文ドラフト・自己推薦文の同時生成 (トークン・レイテンシ削減)
 - PDF / Office ファイルの Markdown 変換 (Workers AI toMarkdown)、Browser Rendering による SPA 参考サイトの取得
- **プロンプト運用**
 - プロンプトのバージョン管理 (20 回超の改修サイクル) と、本番データでの「AI 判定 vs 人間判断」の乖離分析によるデータ駆動の継続改善
 - Claude Code / Codex 向けの自作スキルによる、プロンプトチューニング・未対応案件の一括選別の半自動化
 - AI による実績の捏造を防ぐネガティブリストなどのガードレール設計
- **フルスタック / 運用**

- Hono + oRPC による型安全な API、React 19 + TanStack Router / Query による SPA、Drizzle ORM + D1
- Slack のボタン・モーダルによる判断記録 (HMAC 署名検証)、Cron Triggers による日次リマインド・ステータス監視
- GitHub Actions による CI/CD (D1 マイグレーション自動適用・Worker Secrets 同期)

取り組み・貢献

- 案件選別業務の自動化
 - 案件メールの受信から評価・応募文ドラフト生成までを全自動化し、営業の案件選別を「全件読む」から「判断するだけ」に省力化
 - エントリー送信はあえて自動化せず、人間の最終判断を挟む安全設計とした
- データ駆動のプロンプト改善運用
 - プロンプト改修の履歴・観点をリポジトリに記録し、本番の判定結果と人間の判断の乖離から判定精度を継続的に改善する運用を確立

業務 6: 社外向け資料共有基盤の開発 (Cloudflare Workers)

概要

- 提案書などの社外向け資料を Cloudflare Access 認証付きで顧客に配信する資料共有基盤を、Cloudflare Workers + Static Assets で単独開発・運用
- 案件担当者はディレクトリに資料を置くだけで顧客への URL 共有が完結する仕組み

経験した技術

- Worker の HTMLRewriter による共通フレーム挿入と Markdown の実行時 HTML 化 (ビルドレス設計)
- GitHub App (RS256 JWT + installation token) による別リポジトリ資料の allowlist 型ライブ proxy、Cache API の stale-while-revalidate
- access 設定ファイルを SSoT とした Cloudflare Access ポリシーの GitHub Actions 自動同期
- 誤公開を防ぐ二重の機械的チェック (fail-safe 設計)、機密設定ファイルの配信禁止・ダウンロード機能の非搭載など認証境界の多重防御
- git 履歴からの目次メタデータ自動生成、Cloudflare Observability MCP / Chrome DevTools MCP による本番ログ調査・Access 越しの UI 検証

取り組み・貢献

- 顧客資料の共有を「URL を渡すだけ」に統一し、認証・公開範囲の管理を IaC 的に機械強制することで、非エンジニアの案件担当者でも安全に使える基盤を実現

開発環境

フロントエンド

- Astro 6, React 19, TanStack Router, TanStack Query, shadcn/ui, TypeScript, Tailwind CSS v4, Bun, Astro Content Collections, satori, sharp

バックエンド / AI システム

- Hono, oRPC, Drizzle ORM, Zod, Better Auth, @modelcontextprotocol/sdk (MCP サーバー開発)
- Cloudflare Workers AI (bge-m3, Kimi K2.6, toMarkdown), AI Gateway, Vectorize, Workflows, Queues, Browser Rendering (Puppeteer), Email Routing, HTMLRewriter
- LINE Messaging API, Slack API (Events / ボタン・モーダル), Google Drive API, GitHub App 連携

クラウド / インフラ

- Cloudflare (Workers, Pages, Pages Functions, D1, R2, KV, DNS, Turnstile, Access), GCP (Workload Identity Federation, GCS, IAM), Terraform, tfint, dotenvx, mise

CI/CD

- GitHub Actions, release-please, Wrangler, tfcmt, Renovate

テスト / 品質

- Vitest, Playwright, Storybook, MSW, @axe-core/playwright, oxlint, oxfmt, knip, syncpack, lefthook, commitlint

AI ツール (提案活動・開発)

- v0 (Vercel), Replit, Lovable, Google AI Studio, Google Stitch, Claude Code (Skills / MCP / 定期ルーチン), Codex, Claude Code Managed Agents, Chrome DevTools MCP

分析 / モニタリング

- Google Search Console, PageSpeed Insights, Cloudflare Analytics

開発ツール

- VSCode, Figma

[No.12] ショートドラマアプリ開発 - フロントエンドエンジニア (React Native / Next.js)

チーム体制

- 案件全体人数 : 6名
 - フロントエンド (iOS + Android + Web) : 2名 (担当)

案件概要・担当業務

- 0 → 1 直後のグロースフェーズにおけるショートドラマアプリの開発
- React Native (Expo) による iOS / Android 同時開発を担当し、並行して Next.js による Web 版の開発も実施

経験した技術

- **React Native (Expo) / Next.js**
 - Monorepo 構成による iOS / Android / Web のクロスプラットフォーム開発
 - Expo SDK のメジャーバージョンアップ対応 (v52→v53、v53→v54、v54→v55)
 - Solito によるモバイル (Expo) / Web (Next.js) 間のコード共有
 - Tamagui を用いたプラットフォーム間の UI 統一
 - expo-iap によるアプリ内課金の実装
 - Rive を用いたアニメーションの組み込み
 - Next.js による Web アプリケーション開発
 - TanStack Query によるデータフェッチング・キャッシング
 - Crisp SDK を用いたカスタマーサポート用チャット機能の実装
- **開発プロセス改善**
 - Storybook による UI コンポーネントカタログの作成
 - Maestro によるモバイルアプリでの E2E テストの導入
 - Mise の導入による開発環境のセットアップの簡略化とバージョンの統一
- **AI・MCP活用**
 - Figma MCP を活用した Figma デザインからの実装
 - Storybook でコンポーネントを作成し、Google DevTools MCP / Playwright MCP で Claude Code による検証

- GitHub Actions による Claude Code 自動レビューの仕組みの構築

取り組み・貢献

- **ビジネス指標に基づくグロース施策の企画・実装**
 - ARPU・CTR・LTV などの KPI を CEO と連携しながら追い、ユーザー獲得コスト vs LTV を意識した施策設計を行った
 - ファネル分析で離脱ポイントを特定し、課金導線の改善によりサブスクの契約者数や課金アイテムの購入数を増加させた
 - ガチャ・ミッション機能・無料開放施策などの設計・実装を通じて、主要 KPI の改善に貢献した
 - オファークロール連携を導入し、収益チャネルを拡大した
- **高速な PDCA サイクルの実践**
 - 施策立案から1週間でリリースする高速な PDCA サイクルをスプリントごとに繰り返して回した
 - A/B テストによる効果検証を行い、季節要因や曜日パターンなど複数の変数がある中で施策単体の効果を正しく測定した
 - 効果が見込めない施策は即座に機能削除し、効果がないという事実も意思決定に活かした
- **デザイナーとの協業**
 - 表示内容や画面遷移の導線、コンポーネントの色や配置といった UI/UX の仕様にまで踏み込んで改善案を提案した
 - 仮実装を素早く共有し、デザイナーと議論しながらリリース前に何度もブラッシュアップを重ねた
 - リリースごとの KPI の結果をデザイナーと共有し、データに基づいた改善提案を繰り返すことで提案の精度を高めていった

開発環境

React Native (Expo) / Next.js

- **アーキテクチャ:**
 - Monorepo 構成 (Expo + Next.js)
- **主要ライブラリ:**
 - TanStack Query, ts-proto, Tamagui, Solito, Expo Modules API (Swift / Kotlin)
- **コード品質・テスト:**
 - Storybook, ESLint, Prettier, Vitest, Maestro

Firebase / Google Cloud

- Firebase (Auth, Distribution, Remote Config, Analytics)
- Google Cloud (Cloud Run)

分析ツール

- BytePlus DI, Looker Studio, Adjust

CI/CD

- GitHub Actions

プロジェクト管理

- GitHub Projects, Notion, Linear

開発ツール

- VSCode, Android Studio, Xcode, GitHub Copilot, Claude Code, Codex, Gemini

デザインツール

- Figma
-

[No.11] OSS 開発 - 個人開発 (TypeScript)

概要

- TypeScript 製の OSS ライブラリ開発 (実用的な CLI ツール)

OSS ライブラリ開発

- **ai-chat-md-export:**
 - ChatGPT と Claude のチャット履歴を Markdown ファイルに変換する CLI ツール
 - npm パッケージ, Homebrew パッケージ, GitHub Releases (Windows, macOS, Linux の各プラットフォームに対応した実行バイナリ) でリリース
 - プライバシーファーストで完全ローカル処理、高速な大量データ処理 (数千の会話を数秒で変換)
 - 日付範囲の指定やキーワード検索によるフィルタリング機能の対応
 - 使用技術:
 - アーキテクチャ: Clean Architecture
 - CLI開発: [Commander.js](#), [ora](#), [consola](#), [zod](#)
 - ドキュメント生成: [@ysk8hori/typescript-graph](#), [tsuml2](#)
 - CI/CD: GitHub Actions, [GoReleaser](#), [release-please](#), [release-it](#)
 - コード品質: [Biome](#), [husky](#), [Codecov](#), [Renovate](#), [Knip](#), [commitlint](#), [@elsikora/git-branch-lint](#)
- **mermaid-markdown-wrap:**
 - Mermaid ファイル (.mmd/.mermaid) を Markdown コードブロックでラップする CLI ツール
 - npm パッケージ, GitHub Actions Marketplace での配布
 - 型安全な設定ファイルのサポート (JSON / YAML / JS / TS 対応、TypeScript 向けヘルパー関数提供)
 - `init` コマンドによる対話形式の初期設定機能
 - 使用技術:
 - アーキテクチャ: レイヤードアーキテクチャ
 - CLI開発: [Commander.js](#), [@clack/prompts](#), [cosmiconfig](#), [globby](#), [Valibot](#)
 - ドキュメント生成: [@ysk8hori/typescript-graph](#)
 - CI/CD: GitHub Actions, [release-please](#)
 - コード品質: [act](#), [Biome](#), [husky](#), [Codecov](#), [Renovate](#), [Knip](#), [commitlint](#), [@elsikora/git-branch-lint](#)
- **issue-linker:**
 - GitHubのIssue参照を実際に検証するCLI/GitHub Actionツール
 - npm パッケージ, GitHub Actions Marketplace での配布
 - JavaScript Actionとして実装し、Composite ActionよりもGitHub Actions実行時間を短縮
 - @vercel/nccによるシングルファイルバンドルで依存関係のインストール時間を削減し、Action起動を高速化
 - bun workspaceによるモノレポ構成でCLI・Action・Coreライブラリを管理
 - 単なるパターンマッチングではなくGitHub APIでIssue番号の存在を確認
 - ブランチ名やコミットメッセージごとに最適化された検出パターンを提供
 - 使用技術:
 - アーキテクチャ: レイヤードアーキテクチャ
 - モノレポ管理: [bun workspace](#)
 - CLI開発: [Commander.js](#), [Valibot](#)
 - GitHub連携: [Octokit](#), [simple-git](#), [micromatch](#), [@t3-oss/env-core](#)
 - ビルド: [@vercel/ncc](#)
 - CI/CD: GitHub Actions, [release-please](#)

- コード品質: [Biome](#), [husky](#), [Codecov](#), [Renovate](#), [Knip](#), [commitlint](#), [@elsikora/git-branch-lint](#)
- [readme-i18n-sentinel](#):
 - READMEの翻訳版と原文の構造的ー貫性を検証するCLIツール
 - npm パッケージでの配布
 - glob patternによる翻訳ファイルの自動検出、行番号レベルでの精密な構造比較
 - 使用技術:
 - アーキテクチャ: レイヤードアーキテクチャ
 - CLI開発: [Commander.js](#), [zod](#)
 - 差分検出: [diff](#), [globby](#)
 - ドキュメント生成: [@ysk8hori/typescript-graph](#)
 - CI/CD: GitHub Actions, [release-please](#)
 - コード品質: [Biome](#), [husky](#), [Codecov](#), [Renovate](#), [Knip](#), [commitlint](#), [@elsikora/git-branch-lint](#)

取り組み・貢献

- 実用的な OSS ライブラリの開発・運用
 - 4つの CLI ツールを npm パッケージとして公開し、実際のユーザーに利用される OSS を運用した
 - Homebrew tap や GitHub Actions Marketplace など、複数の配布チャネルを確立した

[No.10] NFT ゲームアプリ開発 - Flutter リードエンジニア (Flutter)

チーム体制

- 体制
 - PdM : 1名
 - PM : 1名
 - デザイナー : 1名
 - サーバーサイドエンジニア : 2名
 - Flutter エンジニア : 1名 (担当)

案件概要・担当業務

- 0 -> 1 フェーズでの NFT ゲームアプリの開発における Flutter エンジニアを担当
- 唯一の Flutter エンジニアとして、アーキテクチャの考案、ライブラリの選定、PM・デザイナー・サーバーサイドチームとの仕様調整、すべての Flutter アプリの実装を担当

経験した技術

- Flutter
 - Riverpod と Hooks を用いた状態管理
 - Google Maps API の活用と google_maps_flutter , geolocator を用いた地図表示と位置情報の取得
 - go_router を用いた画面遷移の実装
 - openapi_generator を用いた API クライアントコードの自動生成
 - dio を用いた HTTP 通信およびインターセプターによる JWT 認証の実装
 - flutter_secure_storage を用いたセキュアなデータ保存
 - permission_handler を用いた位置情報取得、写真撮影、写真フォルダへのアクセスの実装
 - slang による多言語対応
 - pedantic_mono によるコード品質の向上
 - ThemeData よりデザインシステムの実装

- fvm による Flutter バージョンの管理
- 開発プロセス改善
 - Prism を活用した API モックサーバーの構築
 - Lefthook による pre-commit 時の静的解析実行

取り組み・貢献

- 開発体制の改善活動
 - GitHub Projects を活用したスクラムボードを作成し、タスクの進捗状況を可視化した
 - デイリーを開催し、毎日、メンバー間での情報共有と開発プロセスの改善を行った
 - バグの発見から修正までのプロセスを整備して、それをチーム内で運用した
- API インターフェース設計と UI 先行開発
 - サーバーサイドの Pull Request をレビューし、開発段階で API インターフェースの改善点をフィードバックを行った
 - OpenAPI 形式の yaml ファイルから Prism でのモックサーバーでの開発環境を整備して、UI の先行開発を実施した
- デザインシステムの導入
 - デザイナーと協力してデザインシステムを設計し、デザインの一貫性を実現した
 - デザインシステムを Flutter の ThemeData を通じて定義し、UI の実装コードを削減した

開発環境

Flutter

- アーキテクチャ:
 - Riverpod + Hooks による状態管理
- 主要ライブラリ:
 - go_router, dio, slang, permission_handler, flutter_secure_storage, pedantic_mono, freezed, google_maps_flutter, geolocator, openapi_generator, fvm

CI/CD

- GitHub Actions

プロジェクト管理

- GitHub Projects, Notion, Slack

開発ツール

- VSCode, Android Studio, Xcode, GitHub Copilot, ChatGPT

デザインツール

- Figma

[No.9] SNS アプリ開発 - iOS リードエンジニア (Swift)

チーム体制

- 案件全体人数 : 約10名
 - iOS エンジニア : 1名 (担当)

案件概要・担当業務

- 0 → 1 フェーズでの SNS アプリ開発の立ち上げ案件

- 唯一の iOS エンジニアとして、アーキテクチャの考案、ライブラリの選定、CI/CD 環境の構築、PM・デザイナー・サーバーサイドチームとの仕様調整、すべての iOS アプリの実装を担当

経験した技術

- **Swift**
 - Xcode 16 Beta での Strict Concurrency を含む Swift 6 対応
 - AVFoundation を活用した録音/再生の機能実装
 - [WhisperKit](#), [Speech](#) SDK を活用した音声データの文字起こしの実装
- **開発プロセス改善**
 - [Swift OpenAPI Generator](#) による API 通信処理の自動生成の GitHub Actions パイプラインの構築
 - [Swagger UI Action](#) を用いた API 仕様書の自動生成の GitHub Actions パイプラインの構築
 - [tbls](#) を用いた MySQL のテーブル定義書の自動生成の GitHub Actions パイプラインの構築
 - [pixelmatch](#) による View のスナップショットの差分検出の実装

取り組み・貢献

- **アジャイル開発**
 - テスタブルなアーキテクチャの導入:
 - モックにより API のレスポンスを差し替えられるアーキテクチャを導入し、サーバーサイドチームからの API 提供前から View やビジネスロジックの実装を可能にした
 - デバッグ画面の作成:
 - iOS アプリに検証用のデバッグ画面を作成し、新機能や View の早期検証を可能にした
 - Docs as Code の導入:
 - [Swagger UI Action](#) や [tbls](#) によるドキュメント生成方法を調査して、サーバーサイドチームに展開した
- **CI/CD 環境の構築**
 - Xcode Cloud 導入:
 - Xcode Cloud を活用し、Pull Request のマージをトリガーにして TestFlight 配信を自動化した。これにより、新機能の迅速な検証を可能にした
 - API インターフェース変更の自動 Pull Request 作成:
 - OpenAPI (Swagger) 形式での API インターフェースの変更をトリガーに iOS リポジトリへの自動 Pull Request を作成する GitHub Actions 環境を構築した
 - スナップショット差分テスト:
 - View のスナップショット差分テスト環境を構築し、不具合の早期発見を実現した
- **iOS メンバーの増員や引き継ぎを見越した GitHub 管理**
 - ドキュメント整備:
 - 環境構築手順、ライブラリ選定理由、アーキテクチャ、CI/CD 構成図、ブランチ戦略などを README に記載した
 - プロジェクト管理:
 - リリースノート、タグ、マイルストーン、GitHub Projects を整備し、タスクの進捗を時系列で振り返れるように管理した
- **実装・最新技術**
 - 実装:
 - ワイヤフレーム段階でのデザインを基に iOS アプリを実装し、実装の課題や仕様の課題を早期発見し、チームへ共有した
 - コード生成:
 - View 層や UseCase 層のテストコードを含めたボイラープレートコードは [Sourcery](#) や

[Mockolo](#) によって自動生成し、開発効率を高めた

- Swift 5 → Swift 6 への移行:
 - 早い段階から Beta 版 Xcode を用いて Swift 6 への移行を検証し、大きなトラブルなくスムーズに移行を完了した

開発環境

Swift

- アーキテクチャ:
 - Clean Architecture x Swift Package Manager でのマルチモジュール構成
- Swift 標準 SDK & API:
 - SwiftUI, Swift Package Manager, Swift Concurrency, Combine, AVFoundation, Speech, Swift Testing, String Catalogs, Swift OpenAPI Generator

CI/CD

- Xcode Cloud, GitHub Actions, Renovate

プロジェクト管理

- GitHub Projects, Notion, Backlog

デザインツール

- Figma

[No.8] Clean Architecture ベースの新アーキテクチャの考案 - 個人開発 (Swift)

概要

- Clean Architecture ベースの新しいアーキテクチャである「Framework-Independent Architecture (FIA)」の考案
- 考案したアーキテクチャは Clean Architecture のメリットを享受すると同時に、Xcode のビルド時間の短縮することを目的としている

リポジトリ

- **Framework-Independent Architecture (FIA):** FIA の紹介
 - <https://github.com/sugurutakahashi-1234/framework-independent-architecture>
- **FIA Practical Sample:** FIA を採用した実践的なプロジェクト
 - <https://github.com/sugurutakahashi-1234/fia-practical-sample>

スライド

- Framework-Independent Architecture (FIA) - Clean Architecture で iOS アプリを爆速でビルドする -
 - <https://speakerdeck.com/sugurutakahashi/framework-independent-architecture-fia-clean-architecture-de-ios-apuriwobao-su-debirudosuru>

YouTube

- 【Swift】Clean Architecture で iOS アプリを爆速でビルドする方法 Framework-Independent Architecture (FIA) 【クリーンアーキテクチャ】
 - <https://www.youtube.com/watch?v=5blwYSQcL2E>

経験した技術

- Swift

- SwiftUI の Observation による View の更新
 - String Catalogs による多言語対応
 - Swift 6 対応
 - protocol の any → some 対応
 - Strict Concurrency 対応
 - [Sourcery](#) によるボイラープレートコードの自動生成
 - [SnapshotPreviews-iOS](#) による SwiftUI Previews のカタログアプリの作成
 - テスト
 - Swift Testing によるテストコードの記述
 - [PreviewSnapshots](#) による SwiftUI Previews のスナップショットテスト
 - TestPlan による多言語テスト
 - XCTest による UI テスト
 - 開発体験の向上
 - GitHub Actions による Swift-DocC の ホスティング
 - [depermaid](#) による Swift Package Manager の依存関係の Mermaid 生成
-

[No.7] マーケティングリサーチアプリ開発 - iOS リードエンジニア (Swift)

チーム体制

- 案件全体人数 : 約15名
 - iOS エンジニア : 3名 (iOS リードエンジニア担当)

案件概要・担当業務

- スタートアップ企業の 0 → 1 フェーズでのマーケティングリサーチサービスの立ち上げ案件
- toC 向けのコンテンツ配信アプリと、そのアプリ利用者のデータを用いた toB 向けの Web での BI ツールの 2 つサービスで構成されており、その iOS チームのリードエンジニアを担当

経験した技術

- Swift
 - iOS16 以上を対象 OS とした SwiftUI での画面開発
 - Clean Architecture x Swift Package Manager でのマルチモジュール構成の構築
 - Xcode Cloud での CI/CD 環境の構築
 - Protocol Buffers に対応した [SwiftProtobuf](#) のライブラリを用いたデータ連携
 - async/await, AsyncStream, TaskGroup, Actor などを用いた Swift Concurrency による非同期処理のハンドリング
 - [AWS Amplify SDK](#) を用いた Cognito での SMS での認証・認可、AppSync による GraphQL 疎通、Pinpoint によるログイベント送信、S3 とのデータ連携
 - デザインシステムを活用した画面実装
 - AVFoundation を用いた動画の再生
 - ReplayKit を用いた画面のレコーディング
 - 視線や感情の時系列データの Combine を用いたハンドリング
 - JavaScript を用いたアプリ内 WebView のイベントハンドリング
- 開発プロセス改善
 - GitHub Actions によるリリース tag の生成、リリースノートの作成、PR のレビューワー追加、マイルストーン追加、ラベル追加の自動化の Workflow の実装
 - [Renovate](#) によるライブラリの自動更新 PR の作成の環境構築
 - [Periphery](#) による Swift コードの不要なコードの静的解析
 - Swift-DocC による iOS アプリのドメイン層のドキュメント化

- [Mockolo](#) によるテスト用の Mock の自動生成
- GitHub Copilot, ChatGPT の活用

取り組み・貢献

- iOS リードエンジニアとして、0 → 1 フェーズのアプリ開発における、アーキテクチャ・ライブラリの選定、ランチの戦略の設計、リリース手順の確立、CI/CD 環境の構築、iOS チームのスクラムボードの運用の設計を行った
- AWS Amplify SDK や SwiftProtoBuf のライブラリは、チームとしても経験者がいなかったが、先行して挙動を確認するサンプルアプリを作成して、それをチームに展開することで、それらのライブラリを採用することができた
- 視線分析、感情分析の SDK を組み込み、それらの SDK の入れ替えがあっても、影響範囲を最小限にするようなアーキテクチャを検討して、それを実装した
- PdM、デザイナー、サーバーサイド、データ分析チームとコミュニケーションをとって、アプリの仕様の調整や、データ連携のインターフェースの調整を行った
- iOS チーム内の issue チケットの運用管理を担当し、チームメンバーのタスク状況を常に把握して、他のメンバーがタスクを途切れさせないように先回りして行動し続けた

開発環境

Swift

- **アーキテクチャ:**
 - VIPER ベースの Clean Architecture x Swift Package Manager でのマルチモジュール構成
- **Swift 標準 SDK & API:**
 - SwiftUI, Swift Package Manager, Swift Concurrency, Combine, Swift-DocC, AVFoundation, Core ML, WebKit, ReplayKit, Logger
- **サードパーティ製 SDK:**
 - SwiftProtobuf, Firebase, Amplify, Nimble/Quick, LicensesPlugin, PhoneNumberKit, DeviceKit, SwiftFormat, SwiftGen, Lottie, Mockolo, Mint, Periphery

クラウド連携

- **AWS Amplify:**
 - AppSync (GraphQL), Cognito, S3, Pinpoint
- **Firebase:**
 - Crashlytics

CI/CD

- Xcode Cloud, GitHub Actions, Renovate

プロジェクト管理

- GitHub Projects, Notion, Backlog

インターフェース共有

- Protocol Buffers, Swagger

デザインツール

- Figma

[No.6] ファンクラブアプリ開発 - iOS エンジニア (Swift)

チーム体制

- 案件全体人数：約30名

- iOS エンジニア : 5名 (担当)

案件概要・担当業務

- アーティストのファンクラブアプリにおけるスタンプラリー機能および景品交換の機能の開発を行なった
- デザイナーとの仕様の調整、見積もり、実装、レビュー、バグ修正を行なった

経験した技術

- Redux ベースのアーキテクチャ (VueFlux) での状態管理
- デザインシステムに沿った UI の実装

取り組み・貢献

- デザイナー、Android、Web フロントのエンジニアとコミュニケーションを取りながら、プラットフォーム間で仕様に大きな差がでないように開発した

開発環境

Swift

- **アーキテクチャ:**
 - Redux ベースのアーキテクチャ
- **Swift 標準 SDK & API:**
 - UIKit, AVFoundation
- **サードパーティ製 SDK:**
 - Carbon, VueFlux, ReactiveSwift, XcodeGen, Quick/Nimble, APIKit, CocoaPods, Carthage, Lottie

クラウド連携

- **Firebase:**
 - Crashlytics

CI/CD

- CircleCI, Fastlane

プロジェクト管理

- Wrike, Kibela

インターフェース共有

- Protocol Buffers, Swagger

デザインツール

- Figma

[No.5] ドローン制御アプリ開発 - iOS エンジニア (Swift)

チーム体制

- 案件全体人数 : 約15名
 - iOS エンジニア : 6名 (担当)

案件概要・担当業務

- BtoB 向けドローン制御アプリの iOS アプリの開発におけるドローンの飛行の安定性改善、複数社のドローンの対応、画面の開発などを行なった

- アーキテクチャの検討、見積もり、実装、レビュー、バグ修正を行なった

経験した技術

- **Swift**
 - アーキテクチャの検討
 - Clean Architecture での実装
 - SwiftUI・UIKit x Combine を用いた画面実装
 - Swift Concurrency を用いた非同期処理の実装
 - Firebase Crashlytics、Xcode Organizer を用いたバグの原因調査
 - Logger API を用いたログ出力
 - Quick/Nimble ライブラリを用いた可読性の高いテストコードの記述
 - Mock を活用したテストコードの記述
 - iPad サイズ対応のアプリの実装
- **IoT**
 - 外部ライブラリを用いたドローンの制御の Swift での実装
 - PID 制御などの制御工学の理解と適切な制御モデルの Swift での実装
 - RoS(Robot Operating System) 環境の活用

取り組み・貢献

- Clean Architecture を採用したことによって、UI 実装を変更せずに外部ライブラリの差し替えや、レイヤーごとの独立したテストコード記述を実現した
- UIKit や Delegate パターンでの既存実装を、SwiftUI、Combine、Swift Concurrency といった新しい技術でリファクタリングした
- 以下のようなチームの運用の改善に積極的に取り組んだ
 - 見積会の実施
 - レトロスペクティブの実施
 - 開発チーム朝ハドル会の実施
 - プロダクトバックログを開発者が着手可能であることを表す「Ready」の概念の導入
 - Pull Request 提出から Merge までの運用ルールの見直し
 - リリースブランチ運用の見直し
 - デイリー前の Slack リマインダーの設定
 - デイリーでの相談事項の事前エントリー制の導入
 - Firebase Crashlytics 運用の見直し

開発環境

Swift

- **アーキテクチャ:**
 - VIPER ベースの Clean Architecture
- **Swift 標準 SDK & API:**
 - SwiftUI, UIKit, Combine, Swift Concurrency, Logger, MetricKit
- **サードパーティ製 SDK:**
 - Realm, Quick/Nimble, APIKit, CocoaPods, Carthage

クラウド連携

- **Firebase:**
 - Crashlytics, Analytics

CI/CD

- Bitrise, Fastlane

プロジェクト管理

- Zenhub

デザインツール

- Figma
-

[No.4] ショッピングアプリ開発 - Flutter エンジニア (Flutter)

チーム体制

- 案件全体人数 : 2名
 - Flutter エンジニア : 1名 (担当)
 - デザイナー : 1名

案件概要・担当業務

- Flutter での iOS・Android クロスプラットフォーム開発を採用したショッピングアプリのデモアプリの開発を担当
- Flutter でのフロントエンド実装から Firebase を活用したバックエンド実装まで、すべて一人でを行った

経験した技術

- **Flutter**
 - Provider による状態管理
- **Firebase**
 - Authentication による認証
 - Firestore によるデータの永続化、NoSQL DB 設計
 - Storage への画像データの永続化
 - Crashlytics によるクラッシュ報告管理
 - App Distribution による iOS・Android のアプリ配布
 - Analytics による KPI 指標の集計
 - Google Maps API での地図活用
- **CI/CD**
 - Codemagic での iOS・Android のアプリ配布の自動化
 - Fastlane から App Distribution への配布

取り組み・貢献

- Firebase を活用したサーバーレス構成でのモバイルバックエンドサービスの設計・実装を行った
- Flutter による Android 対応 (マテリアルデザイン、Google Play ストアでの配信) を経験した

開発環境

Flutter

- Provider

クラウド連携

- Firebase:
 - Authentication, Firestore, Storage, Crashlytics, App Distribution, Analytics
- Google Maps API

CI/CD

- Codemagic, Fastlane

デザインツール

- Adobe XD
-

[No.3] 飲食店管理アプリ開発 - iOS リードエンジニア (Swift)

チーム体制

- 案件全体人数：約10名
 - iOS エンジニア：3名（リードエンジニア担当）

案件概要・担当業務

- BtoB 向け飲食店管理モバイルアプリの MVP アプリの作成
- iOS リードエンジニアとして、要件の調整やサーバーサイドチームとの API インターフェースの検討などを行った
- Scrum Master も兼任した

経験した技術

- **Swift**
 - SwiftUI での画面実装
 - Codable プロトコルを用いた JSON の変換
 - TestFlight によるアプリ配信
 - アーキテクチャ、ディレクトリ構成の検討
- **Scrum Master**
 - スクラムボードの設計
 - 会議のファシリテーション

取り組み・貢献

- 決められた要件をただ実装するだけではなく、お客様やデザイナーによりよい仕様やデザインを提案した
- スクラムボードのレーンの扱い、コードレビュー方法、issue の起票方法などについて、レトロスペクティブの場だけでなく、チーム内で相談し、常に運用の改善を行った

開発環境

Swift

- アーキテクチャ：
 - MVVM
- **Swift 標準 SDK & API:**
 - SwiftUI
- サードパーティ製 SDK:
 - SwiftLint

プロジェクト管理

- Zenhub, Trello

デザインツール

- Adobe XD
-

[No.2] クレジットカードアプリ iOS アプリ開発 - iOS エンジニア (Swift)

チーム体制

- 案件全体人数：約20名
 - iOS エンジニア：4-5名（担当）

案件概要・担当業務

- BtoC 向けのクレジットカードアプリの iOS アプリの開発における見積もり、実装、テスト、レビュー、バグ修正を担当した
- メインはサーバーサイドチームの担当であったが、作業の手が空いたり、iOS チームの負荷が上がったときに iOS チームを担当した

経験した技術

- UIKit での画面実装
- API 疎通
- Realm でのデータ永続化
- XCTest でのテストコード実装
- MVVM での実装
- Delegate パターンの実装
- Human Interface Guidelines に基づいた UI 実装
- Moneytree LINK SDK といったサードパーティー製のライブラリの組み込み

開発環境

Swift

- アーキテクチャ：
 - MVVM
- Swift 標準 SDK & API:
 - UIKit
- サードパーティ製 SDK:
 - CocoaPods, Carthage, Realm, Moneytree LINK SDK

通信キャプチャ

- mitmproxy

プロジェクト管理

- Jira, Confluence, Trello

デザインツール

- Sketch, InVision

[No.1] クレジットカードアプリ API 開発 - サーバーサイドエンジニア（PHP）

チーム体制

- 案件全体人数：約20名
 - サーバーサイドエンジニア：4名（担当）

案件概要・担当業務

- BtoC 向けクレジットカード明細管理アプリのリニューアルに伴い、API やバッチの開発
- サプリードディベロッパーとして、お客様向け説明資料の作成、設計、見積もり、実装、テスト、レビューを担当

- アジャイル開発を採用しており、サーバーサイドチーム結成から約 2 年半に渡り、リリースしたシステムについて継続的にアップデートを行なった

経験した技術

- **API 設計/開発**
 - PHP での API 設計・開発
 - MySQL での DB 設計・開発
 - OAuth2.0 での認証・認可の実装
- **バッチ設計/開発**
 - Java でのバッチの設計・開発
- **テスト**
 - API の単体・結合テストの設計と実装
 - Postman での API テストの自動化
 - JMeter での負荷テスト
- **ドキュメンテーション**
 - OpenAPI (Swagger) でのインターフェース設計・共有
 - PlantUML での設計

取り組み・貢献

- モバイルアプリケーションのバックエンド開発における設計、実装、テスト、リリース、運用までのフルライフサイクルを経験できた
- iOS チームと兼任していたため、モバイルアプリからの視点を API のインターフェースの設計に取り込むことができた
- API の結合テストを Postman によって自動化することで、少ない工数で網羅的に繰り返しテストを実施し、品質を担保することができた

開発環境

使用言語

- PHP (CodeIgniter)
- Java

プロジェクト管理

- Jira, Confluence, Trello

テストツール

- Postman, JMeter

ドキュメンテーション

- OpenAPI, PlantUML, draw.io
-